Today 2:28 AM

I found these libraries online, and they're really useful. But I want to create, like, a demo, or, like, an ideal case where I use all these libraries all at once. What dataset can I use in order to showcase the utility of each of these libraries?

Data Analysis, Profiling & Utility Libraries
D-Tale: man-group/dtale
pandas-dq: AutoViML/pandas_dq
klib: klib / data cleaning
SweetViz: fbdesignpro/sweetviz
RapidFuzz: rapidfuzz/RapidFuzz
Missingno: Fasterius/missingno
Visualization Libraries (Joseph Barbier / y-sunflower)
Pypalettes: y-sunflower/pypalettes
DrawArrow: y-sunflower/drawarrow
PyFonts: y-sunflower/pyfonts
Dayplot: y-sunflower/dayplot
MoreThemes: y-sunflower/morethemes
BumpPlot: y-sunflower/bumplot
NineJS: y-sunflower/ninejs
Folium - turn location data into interactive maps
• Great Tables - make clean, presentation-ready tables without fixing everything in Excel *
• PandasGUI - explore, filter and visualize DataFrames through an interactive GUI L
• UpSetPlot - visualize overlaps across multiple customer groups or segments
• CleverCSV - automatically detect messy CSV structures and delimiters when read_sv () fails

Show less

Yes — and I'd actually **design the dataset around the libraries**, rather than pick a generic dataset and then force the libraries onto it.

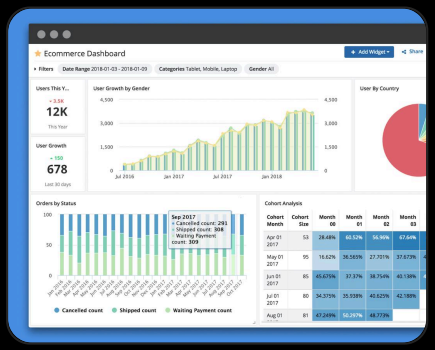
The ideal showcase dataset would have:

- **messy raw CSVs** → CleverCSV
- **missing values and data-quality problems** → pandas-dq, Missingno, Klib
- **duplicate / inconsistent names** → RapidFuzz
- **multiple customer segments** → UpSetPlot
- **dates over time** → Dayplot
- **geographical information** → Folium
- **categorical + numerical + text columns** → D-Tale, SweetViz, PandasGUI
- **enough variables for statistical exploration** → Great Tables
- **a compelling visual story** → the y-sunflower visualisation libraries
- ideally **multiple related tables** so you're demonstrating something resembling a real analytics workflow rather than a toy dataframe.

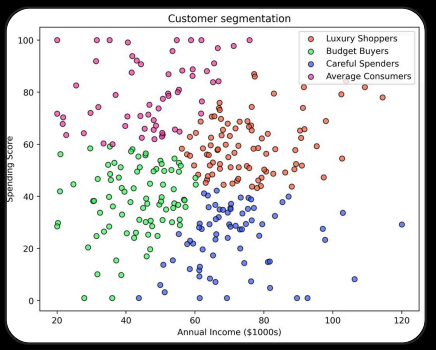
My strongest recommendation: an e-commerce / customer analytics dataset

I'd build a **synthetic "Global E-Commerce & Customer Behaviour" dataset**.

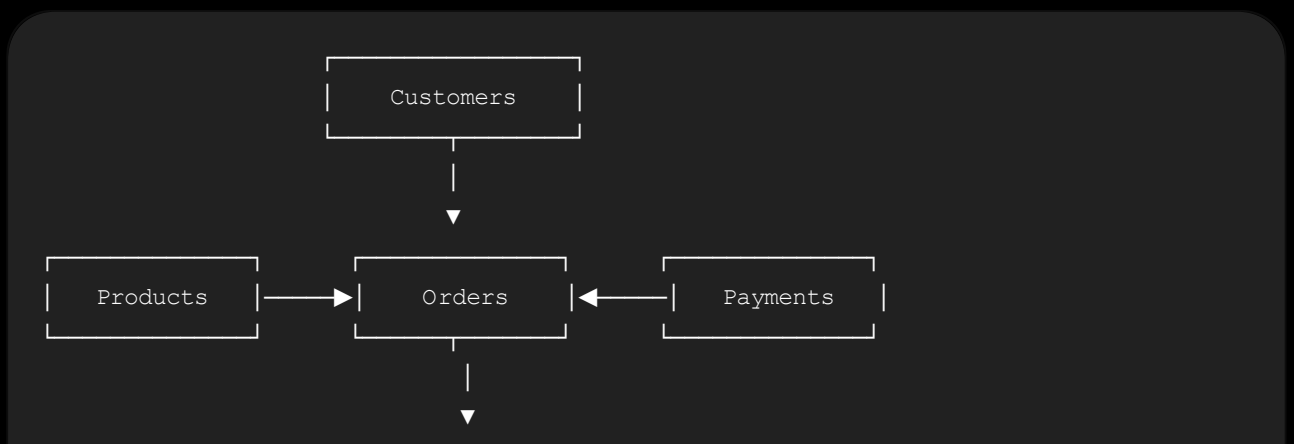
Think of it as a company similar to Amazon/Shopify/ASOS with customers, orders, products, marketing interactions and locations.

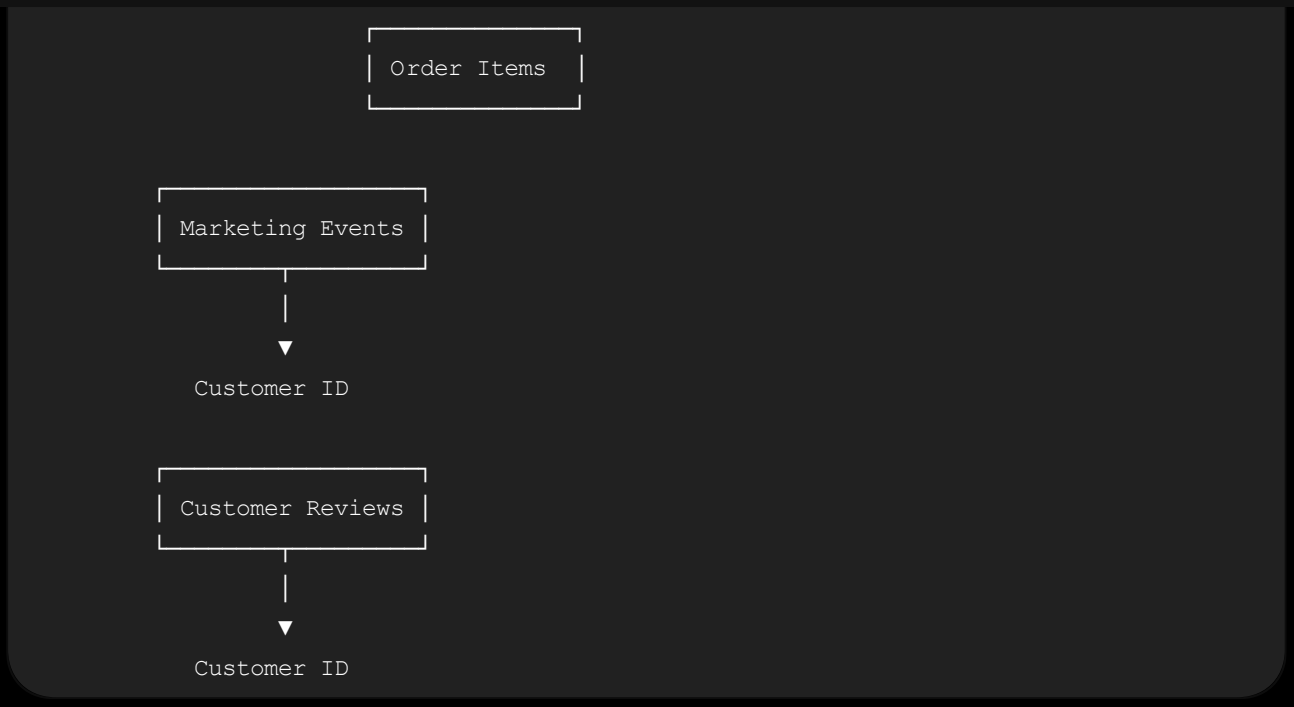






A good architecture would be:





That gives you **multiple dimensions of messiness and analysis** rather than just one flat CSV.

The dataset I'd actually create

I'd make roughly **150,000–300,000 customers** and perhaps **1–2 million orders**.

You don't necessarily need to work with the entire thing interactively — you could create a representative 100k-row analytical extract.

`customers.csv`

Column	Example
customer_id	C001928
first_name	George
last_name	Smith
email	george.smith@gmail.com
country	United Kingdom
city	London
postcode	SW1A 1AA
latitude	51.5074

Column	Example
longitude	-0.1278
signup_date	2024-02-14
acquisition_channel	Paid Search
customer_segment	Premium
age	31
gender	Male
income_band	£40k–£60k

But deliberately introduce problems:

```
United Kingdom
UK
U.K.
United kingdom
United Kingdom
GB
Great Britain
```

That gives **RapidFuzz** something meaningful to do.

`orders.csv`

```
order_id
customer_id
order_date
product_id
quantity
unit_price
discount
shipping_cost
payment_method
order_status
delivery_days
```

And introduce:

- missing delivery dates
- impossible quantities

- negative discounts
- duplicate orders
- inconsistent dates
- malformed IDs
- outliers
- missing payment methods.

Now you've got a playground for:

pandas-dq + Missingno + Klib + D-Tale + SweetViz

`products.csv`

```
product_id
product_name
category
subcategory
brand
supplier
cost_price
selling_price
rating
review_count
```

You can deliberately make product names messy:

```
Apple AirPods Pro
Apple AirPods Pro
apple airpods pro
Apple AirPods Pro (2nd Gen)
APPLE AIRPODS PRO
AirPods Pro
```

Again: **RapidFuzz**.

`marketing.csv`

```
customer_id
campaign_id
channel
campaign
impressions
clicks
email_opened
```

```
conversion
campaign_date
```

This lets you demonstrate customer overlap:

> Customers who received email AND clicked paid search AND purchased.

That's where **UpSetPlot** becomes genuinely useful.

`reviews.csv`

This is the one I'd definitely include.

```
review_id
customer_id
product_id
review_date
rating
review_title
review_text
verified_purchase
```

Now you have:

- numerical data
- categorical data
- dates
- free text
- missingness
- duplicates
- customer relationships.

It makes the dataset feel much more like **real business data**.

How I'd use every library

This is where the project becomes really good.

Library	Demonstration
CleverCSV	Automatically identify delimiter/structure in deliberately messy CSV files
pandas-dq	Automated data-quality assessment

Library	Demonstration
Missingno	Visualise missing-data patterns
Klib	Cleaning + exploratory analysis + feature inspection
D-Tale	Interactive exploration of individual DataFrames
SweetViz	Automated EDA report comparing datasets/segments
RapidFuzz	Fuzzy matching customers/products/countries
PandasGUI	GUI-based filtering, sorting and exploration
Great Tables	Produce polished KPI/customer-segment tables
UpSetPlot	Customer overlap between marketing channels
Folium	Customer/order density by geography
PyPalettes	Generate coherent visual palettes
DrawArrow	Annotate important trends/events
PyFonts	Typography in visualisations
Dayplot	Analyse temporal activity patterns
MoreThemes	Style visualisations consistently
BumpPlot	Rank movement of product categories/brands
NineJS	Experimental/alternative visual presentation
BumpPlot	Category/brand ranking over time

The important thing is that **each library has an actual reason to exist**.

You're not doing:

> "Here's a chart because I found a cool chart library."

You're doing:

> "This is the analytical problem, and this library is particularly useful for

> solving/presenting it."

That's a much stronger portfolio demonstration.

The really good part: deliberately create a "bad" dataset

This is what I'd do differently from most portfolio projects.

Don't generate a perfectly clean dataset.

Create:



```
                    ↓
Custom visuals
```

That gives you an **actual data engineering → data quality → analytics → visualisation pipeline**.

I'd give the project a business question too

Otherwise it risks becoming:

> "Look at all these Python libraries I know."

Instead:

E-Commerce Customer Intelligence & Data Quality Lab

Business questions

Customer

- Who are our highest-value customers?
- Which customers are at risk of becoming inactive?
- How does customer value vary geographically?

Marketing

- Which acquisition channels produce the highest-value customers?
- How much overlap exists between marketing channels?
- Which campaigns generate repeat purchasers?

Products

- Which products/categories are gaining or losing rank?
- Which products have high sales but poor ratings?
- Which brands dominate different customer segments?

Operations

- Where are delivery times highest?
- Are delivery delays associated with customer churn?
- Which geographical regions generate the most revenue?

Data quality

- Which source contains the most missing data?
- How many customer records can be resolved through fuzzy matching?
- How much does cleaning change the analytical results?

That last category is particularly interesting.

And there's an even better twist

Have **two versions of the data**:

`raw/`

Intentionally disgusting.

```
customers_01.csv
customers_02.csv
orders_2024.csv
orders_2025.csv
orders_export_final.csv
products.csv
marketing_export.csv
reviews.csv
```

Different:

- delimiters
- encodings
- date formats
- column names
- missing-value representations
- whitespace
- spelling
- duplicate records.

Then:

`processed/`

```
dim_customer.parquet
dim_product.parquet
dim_date.parquet
fact_orders.parquet
fact_marketing.parquet
fact_reviews.parquet
```

Now you're essentially building a **miniature analytical data platform**.

And given your existing work with Power BI, Snowflake-style schemas, ETL and predictive modelling, this would actually complement your portfolio extremely well rather than being another generic Kaggle notebook.

One thing I'd change about your library list

I'd **not force every single one into the main analysis**.

Some of the `y-sunflower` libraries are primarily about visual style/experimentation. I'd create a dedicated:

`05_visualisation_showcase.ipynb`

where you take **the same analytical results** and render them using different visualisation approaches.

For example:

```
Question:
"How did brand rankings change over 2024–2026?"

              ↓

         BumpPlot
              ↓
        Traditional
         Matplotlib
              ↓
        MoreThemes
              ↓
         PyPalettes
              ↓
         PyFonts
              ↓
        DrawArrow
```

That demonstrates something much more valuable:

You understand visualisation as a communication problem, not just as a library API.

My ideal project structure

```
ecommerce-intelligence-lab/
│
├── data/
│   ├── raw/
│   │   ├── customers/
│   │   ├── orders/
│   │   ├── products/
│   │   ├── marketing/
│   │   └── reviews/
│   │
│   ├── processed/
```

```
│   └── reference/
│
├── notebooks/
│   ├── 01_ingestion_clevercsv.ipynb
│   ├── 02_data_quality.ipynb
│   ├── 03_entity_resolution.ipynb
│   ├── 04_eda.ipynb
│   ├── 05_customer_analysis.ipynb
│   ├── 06_marketing_analysis.ipynb
│   ├── 07_product_analysis.ipynb
│   └── 08_visualisation_showcase.ipynb
│
├── src/
│   ├── ingestion/
│   ├── cleaning/
│   ├── matching/
│   ├── analysis/
│   └── visualisation/
│
├── reports/
│   ├── sweetviz/
│   ├── missingness/
│   └── data_quality/
│
├── requirements.txt
├── README.md
└── pyproject.toml
```

And the README can literally have a **"Library → Problem → Demonstration"** table.

That would turn this from a collection of cool Python libraries into a **proper portfolio project demonstrating data ingestion, data quality, entity resolution, EDA, analytical modelling and advanced visualisation in one coherent workflow.**

If you want the *best possible* version of this, I'd actually generate the **synthetic dataset itself with controlled defects** (e.g. exactly how many duplicates, missing values, fuzzy matches, outliers, malformed CSVs, geographic records, temporal patterns, etc.), so every library has a deliberately engineered problem to solve.

Can it combined into this repo https://github.com/garcane/End-to-End-LLM-Powered-Customer-Intelligence-Pipeline

A réfléchi quelques secondes

Yes — **and actually, I think that repo is a much better home for this idea than creating a separate project.**

I checked the repository. It is already positioned as an end-to-end **membership/customer intelligence platform**, with ETL, PostgreSQL, feature engineering, ML, data-quality monitoring, segmentation, an LLM insight layer and a dashboard.

So rather than creating an unrelated "Python Libraries Showcase", I'd turn this into a **Data Intelligence Engineering Lab inside the existing pipeline**.

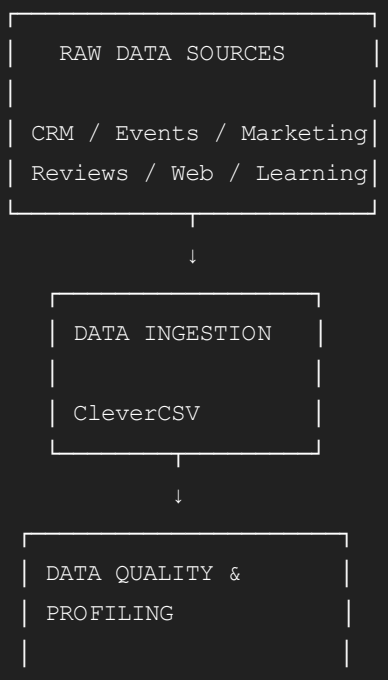
Your GitHub repository

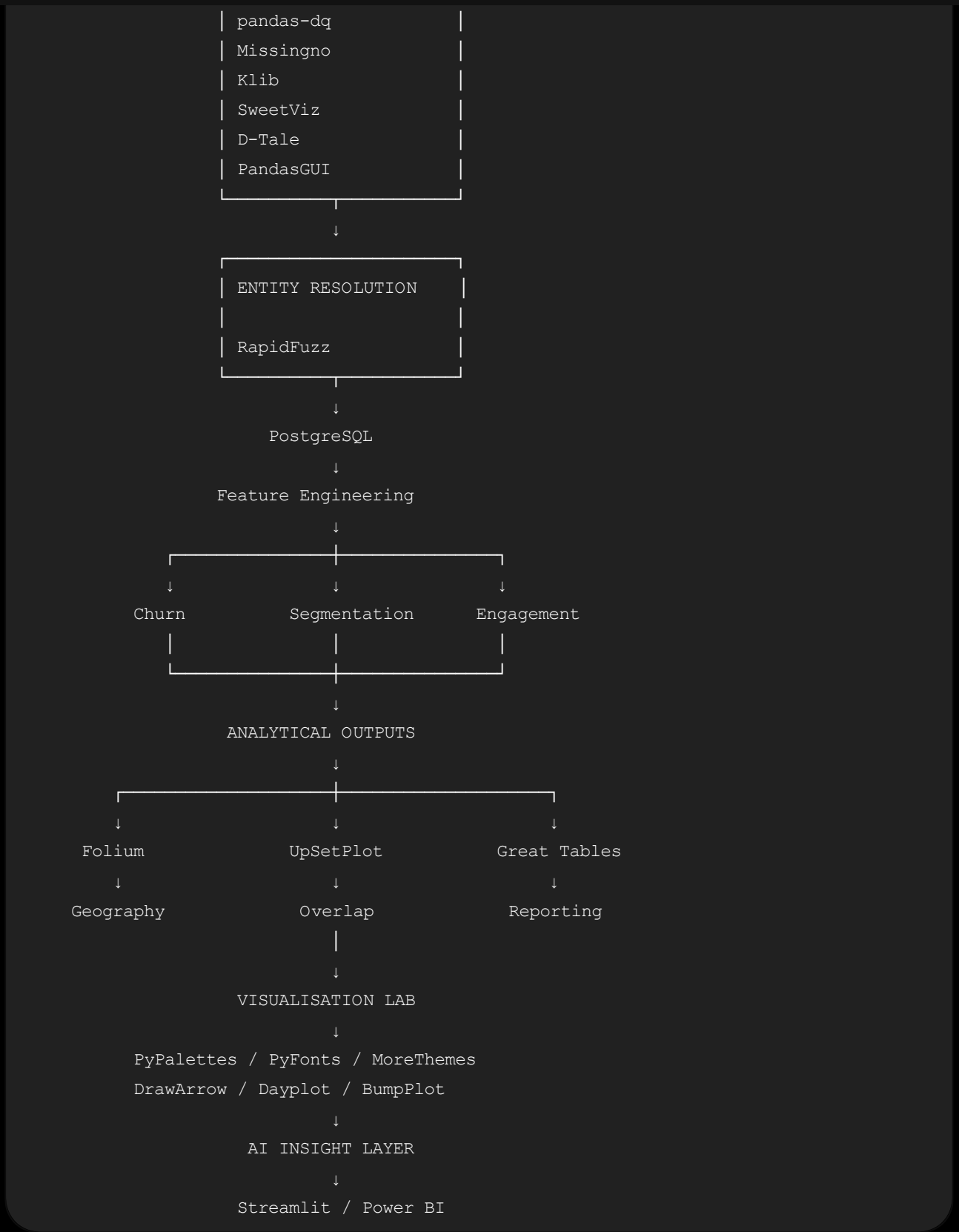
The key idea

Your existing architecture is essentially:

```
Raw Membership Data
        ↓
      ETL
        ↓
   PostgreSQL
        ↓
Feature Engineering
        ↓
 ML / Segmentation
        ↓
   AI Insight Layer
        ↓
    Dashboard
```

I'd expand it to:





That is **much more impressive**.

And I would change the dataset slightly

I wouldn't use generic e-commerce anymore.

Your repo is explicitly about **membership intelligence**. The dataset should therefore simulate a large professional membership organisation.

Something like:

`members.csv`

```
member_id
first_name
last_name
email
country
city
postcode
latitude
longitude
date_joined
membership_type
membership_status
profession
specialism
age
organisation
```

`events.csv`

```
event_id
event_name
event_type
event_date
location
capacity
```

`event_attendance.csv`

```
member_id
event_id
registration_date
attendance_status
```

`learning_activity.csv`

```
member_id
course_id
activity_date
activity_type
```

```
duration_minutes
```

`marketing.csv`

```
member_id
campaign_id
channel
sent_date
opened
clicked
converted
```

`website_activity.csv`

```
member_id
timestamp
page
session_id
duration_seconds
```

`feedback.csv`

```
feedback_id
member_id
date
rating
feedback_text
category
```

`membership_transactions.csv`

```
member_id
transaction_date
transaction_type
amount
payment_status
```

Now you have the exact ingredients your existing project is already designed around.

Your README already talks about CRM, events, marketing, website activity and feedback as sources.

The really clever bit: make the raw data deliberately awful

This is where I think the project could become **properly excellent**.

Your raw data shouldn't be clean.

For example:

Country

```
United Kingdom
UK
U.K.
United kingdom
GB
Great Britain
```

Membership status

```
Active
active
ACTIVE
Actv
Active Member
```

Member names

```
John Smith
John  Smith
john smith
Jon Smith
J. Smith
```

Dates

```
2025-01-03
03/01/2025
01/03/2025
03-Jan-25
2025/01/03
```

Missing values

```
NULL
N/A
NA
Unknown
-
```

```
?
""
```

And then:

CleverCSV discovers the file structure.

pandas-dq identifies the quality problems.

Missingno visualises the missingness.

Klib helps profile/clean the dataset.

RapidFuzz resolves entities.

SweetViz produces the automated EDA.

D-Tale/PandasGUI let you interactively investigate suspicious records.

That gives each library a **real job**.

Then the LLM becomes much more interesting

This is the part I'd particularly like to see in your repo.

Instead of:

> "LLM, analyse this dataset."

You create a chain like:

```
Messy Raw Data
      ↓
Data Quality Engine
      ↓
Clean Analytical Dataset
      ↓
Statistical Analysis
      ↓
ML Models
      ↓
Business Metrics
      ↓
LLM
      ↓
Executive Insight
```

So the LLM receives something like:

JSON

```
{
  "segment": "Event-Focused Members",
  "members": 8421,
  "retention_rate": 0.87,
  "avg_events_attended": 4.2,
  "avg_learning_hours": 8.7,
  "churn_rate": 0.08,
  "top_risk_factor": "declining event attendance",
  "top_retention_driver": "event participation"
}
```

And then generates:

> Event-focused members demonstrate substantially stronger retention than the wider membership base. Members attending four or more events annually show the strongest retention profile, while declining event participation appears to be an early indicator of disengagement.

That's legitimate AI-assisted analytics.

The README already describes the intended AI layer as transforming analytical outputs into business recommendations and plain-English summaries.

Then your visualisation libraries become the presentation layer

This is where I'd have some fun.

Imagine discovering:

> "Members who attend events are significantly more likely to renew."

You could produce:

Standard analytical visual

Matplotlib / Plotly

Temporal behaviour

Dayplot

> How does engagement change throughout the year?

Segment ranking

BumpPlot

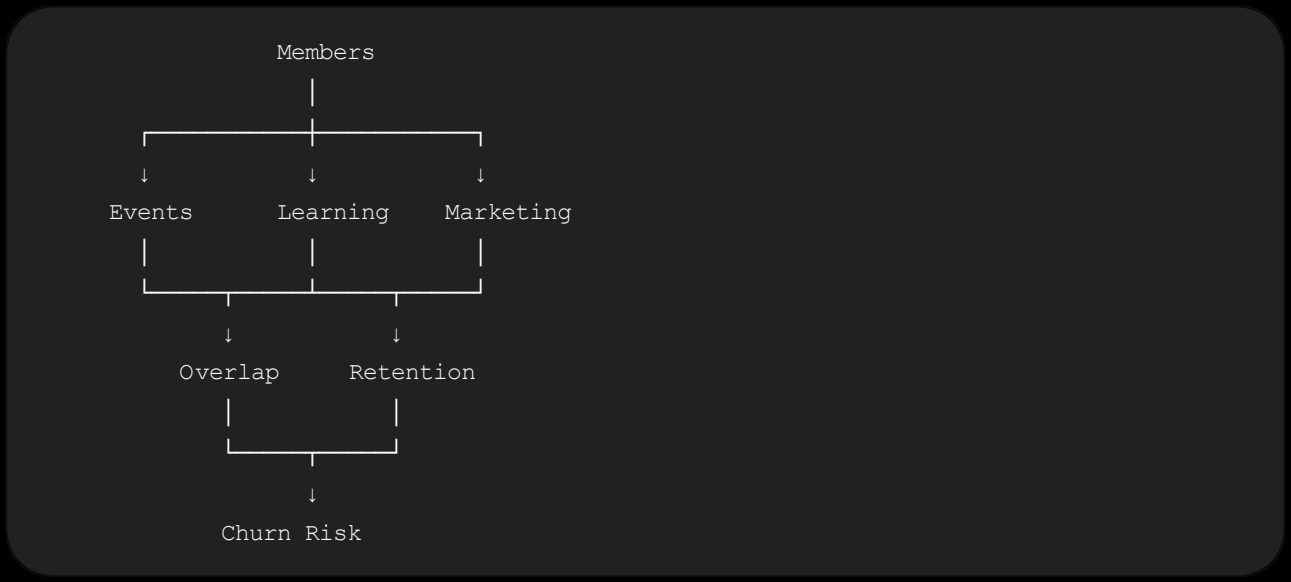
> How have membership segments changed in relative importance?

Overlap

UpSetPlot

> Members who attend events + use learning + engage with marketing.

Something like:



Geography

Folium

Map:

- member concentration
- event attendance
- churn risk
- engagement
- regional penetration.

Executive tables

Great Tables

Produce something like:

Segment	Members	Retention	Churn Risk	Avg Events
Highly Engaged	8,421	91.4%	4.2%	5.8
Event Focused	6,203	87.1%	8.1%	4.2
Passive	12,812	68.3%	24.7%	0.7
At Risk	4,921	41.8%	58.2%	0.2

That's far more compelling than simply showing that you know `great_tables`.

I'd actually make one major addition to the repo

Create:

```
notebooks/
│
├── 01_data_ingestion.ipynb
├── 02_data_quality.ipynb
├── 03_entity_resolution.ipynb
├── 04_exploratory_analysis.ipynb
├── 05_membership_intelligence.ipynb
├── 06_churn_modelling.ipynb
├── 07_customer_segmentation.ipynb
├── 08_visualisation_lab.ipynb
└── 09_llm_insight_generation.ipynb
```

And then:

```
src/
├── ingestion/
├── quality/
├── cleaning/
├── entity_resolution/
├── features/
├── models/
├── visualisation/
└── llm/
```

That turns the repo into something closer to a **miniature analytics platform**.

And there's a particularly nice portfolio story here

Your existing README says the project is intended to demonstrate data engineering, database design, analytics engineering, machine learning, prompt engineering and application deployment.

Adding this library layer gives you another dimension:

> **Data Quality Engineering + Analytical Tooling + Visual Analytics**

So instead of your CV saying:

> "Built an LLM-powered customer intelligence pipeline."

You can legitimately say:

> **Built an end-to-end customer intelligence platform incorporating automated data-quality profiling, entity resolution, exploratory analysis, predictive modelling, customer segmentation, geospatial analytics, advanced visualisation and LLM-generated business recommendations.**

That's a **much stronger project**.

And importantly, you aren't bolting random libraries onto an existing project. **The libraries become components of the intelligence pipeline.**

I would absolutely combine them into this repo.

That's fine by me

Yeah — then I'd keep it as **one repo** and treat the library expansion as a deliberate evolution of the existing project rather than a separate showcase.

The end state is basically:

messy data → ingestion → data quality → entity resolution → analytics → ML → visualisation → LLM insights → dashboard

That gives you a genuinely end-to-end portfolio project rather than a collection of disconnected notebooks.

ChatGPT can make mistakes. Check important info.